Classical Planning: Representation and Relaxation

Delete Relaxation in FDR and the FF Heuristic

Machine Intelligence - Lecture 4 (Slides 18-42)
Daniel Fišer, Fall 2025

Contents

1 Introduction: Heuristics in Planning **2**

1.1 Motivation for Heuristics in Planning . . . 2

1.2 Key Properties of Heuristics . . . 2

1.3 The Challenge: Domain-Independent Heuristics . . . 2

2 Delete Relaxation in FDR **3**

2.1 Intuition: What is Delete Relaxation? . . . 3

2.2 Formal Definition: Delete-Relaxed State Space . . . 4

2.3 Delete-Relaxed Plans and Heuristic h^+ . . . 5

2.4 Computational Complexity of h^+ . . . 6

3 Computing Delete-Relaxation Heuristics **6**

3.1 Relaxed Planning Graph (RPG) . . . 6

3.2 Heuristic Estimates from RPG . . . 7

3.3 Simple Delete Relaxation Heuristic Algorithm . . . 8

3.3.1 HOW TO EXECUTE THIS ALGORITHM MANUALLY (FOR EXAMS!) . . . 9

4 The FF Heuristic **12**

4.1 Motivation for FF . . . 12

4.2 FF Algorithm Structure . . . 13

4.3 FF Algorithm - Detailed Pseudocode . . . 14

4.3.1 HOW TO EXECUTE FF HEURISTIC MANUALLY (FOR EXAMS!) . . . 16

4.4 FF Heuristic Example: Complete Worked Example with Tables . . . 18

4.5 FF Heuristic Example: Blocks World . . . 20

4.6 Properties of FF Heuristic . . . 20

4.7 Why FF is Not Admissible . . . 21

5 Additional Delete-Relaxation Heuristics **22**

5.1 Comparison of Heuristics 22
5.2 LM-Cut Heuristic (Advanced) 22
5.3 Choosing the Right Heuristic 23

6 Practical Considerations and Extensions **23**
6.1 Weighted A* with Non-Admissible Heuristics 23
6.2 Using FF in Practice: The FF Planner 23
6.3 Limitations of Delete Relaxation 24
6.4 Extensions: Beyond Simple Delete Relaxation 25

7 Summary and Key Takeaways **25**
7.1 Main Concepts 25
7.2 Formulas and Notation Reference 26
7.3 Algorithm Complexity Summary 26
7.4 Practical Guidelines 27
7.5 Connection to Previous and Future Topics 27

1 Introduction: Heuristics in Planning

1.1 Motivation for Heuristics in Planning

The problem: Remember how planning is PSPACE-complete? That's computer science speak for "really, really hard!" Without help, finding plans would take forever.

The solution: Use **heuristics** - smart guesses about which direction to search!

Simple analogy: Imagine you're lost in a forest trying to find a lake. Without a heuristic, you'd wander randomly. With a heuristic (like "walk downhill - water flows down"), you'd find it much faster!

In classical planning with FDR (Finite Domain Representation), we saw that the plan existence problem is **PSPACE-complete**, which means finding plans is computationally hard. To make planning practical, we need **heuristic functions** that guide the search toward goal states efficiently.

Heuristic Function in Planning

A **heuristic function** for a planning task $\Pi = (V, A, \text{cost}, I, G)$ is a mapping:

$$h : \Theta_\Pi \to \mathbb{N}_0 \cup \{\infty\}$$

where $h(s)$ estimates the cost of reaching a goal state from state s.
Goal states: States s where $G \subseteq s$, so $h(s) = 0$ for goal states.
Perfect heuristic: $h^*(s)$ denotes the *actual* minimum cost from s to any goal state.

1.2 Key Properties of Heuristics

Admissibility

A heuristic h is **admissible** if:

$$h(s) \leq h^*(s) \quad \text{for all states } s \in \Theta_\Pi$$

That is, h **never overestimates** the true cost to reach a goal.
Why it matters: Admissible heuristics guarantee that A* search finds optimal plans.

Consistency (Monotonicity)

A heuristic h is **consistent** if for every state s and every action a applicable in s:

$$h(s) \leq \text{cost}(a) + h(s[a])$$

where $s[a]$ is the successor state after applying action a.
Relationship: Consistent $\Rightarrow$ Admissible (but not vice versa)

1.3 The Challenge: Domain-Independent Heuristics

In domain-specific planning, we could hand-craft heuristics for each problem (e.g., straight-line distance for route planning). However, in **domain-independent planning**, we want heuristics that work for *any* FDR planning task without manual design.

The Delete Relaxation Approach

Delete relaxation is a systematic technique for automatically deriving admissible heuristics from the structure of FDR planning tasks. The key idea:

1. **Relax** the planning task by removing the "delete" effects of actions
2. Solve (or approximate) the relaxed problem
3. Use the cost of the relaxed solution as a heuristic for the original problem

This yields **domain-independent admissible heuristics** that work for any FDR task.

2 Delete Relaxation in FDR

2.1 Intuition: What is Delete Relaxation?

The BIG idea in one sentence: What if things you achieve never go away?

Real-world analogy: Imagine playing a video game where once you unlock an achievement, it stays unlocked forever - even if you "lose" it would normally disappear. That's delete relaxation!

In FDR planning, actions have two types of effects:

- **Positive effects:** Facts that become true (variables get new values) - Things you GAIN
- **Delete effects:** Facts that become false (variables lose their old values) - Things you LOSE

Delete relaxation simplifies planning by *ignoring delete effects*. Once a fact becomes true, it stays true forever, even if actions would normally change it.

Why is this useful? It makes planning easier! In the real world, moving from A to B means you're no longer at A (delete effect). In the relaxed world, you can be at both A and B simultaneously (unrealistic but useful for estimates)!

Intuitive Example: Truck Logistics

Scenario: You have a truck that needs to move from City A to City B.
Original problem (REAL WORLD):

- Variable: $\textit{truck-loc} \in \{A, B, C\}$
- Action: $\textit{drive_A_to_B}$ - "Drive the truck from A to B"
  - Precondition: $\textit{truck-loc} = A$ - "Truck must be in A first"
  - Effect: $\textit{truck-loc} = B$ - "Truck moves to B"
  - Delete effect: $\textit{truck-loc} = A$ becomes false - "Truck is NO LONGER in A"
- **This makes sense:** A truck can only be in one place at a time!

Delete-relaxed problem (RELAXED WORLD):

- Action: $\textit{drive_A_to_B}$ (relaxed version)
  - Precondition: $\textit{truck-loc} = A$ - "Truck must be in A first" (SAME)

- Effect: *truck-loc* $= B$ - "Truck moves to B" (SAME)
- No delete: *truck-loc* $= A$ *remains true*! - "Truck STILL in A too!"

- After applying the action, both *truck-loc* $= A$ and *truck-loc* $= B$ are true simultaneously
- **Mind-bending but useful:** The truck is magically in BOTH places at once!

Consequence: In the relaxed world, the truck can be in multiple locations at once! This is physically impossible, but it makes planning easier.
Why this helps: Imagine you need the truck in both A and C eventually. In the real world, this requires careful planning and backtracking. In the relaxed world, once you've visited both, you "have" both locations available - making it easier to estimate how many moves you need!

Why Delete Relaxation Helps

Benefits:

1. **Monotonicity:** Facts never become false, so progress is always preserved
2. **Easier to solve:** Delete-relaxed planning can be solved in polynomial time
3. **Admissible:** Any solution to the relaxed problem provides a lower bound on the cost for the original problem

Trade-off: The relaxed problem is easier but may be unrealistic (e.g., truck in multiple places).

2.2 Formal Definition: Delete-Relaxed State Space

Delete-Relaxed FDR Planning Task

Given an FDR planning task $\Pi = (V, A, \text{cost}, I, G)$, the **delete-relaxed task** Π^+ modifies how states work:
Delete-relaxed states: Instead of complete variable assignments, states in Π^+ are *sets of facts* (partial states) where:

- A state s^+ is a set of facts: $s^+ \subseteq \{(v, d) \mid v \in V, d \in D_v\}$
- Multiple facts about the same variable can coexist: $(v, d_1) \in s^+$ and $(v, d_2) \in s^+$ for $d_1 \neq d_2$

Initial state: $I^+ = I$ (same as original, but now treated as a set of facts)
Goal: $G^+ = G$ (same partial state condition)
Actions: Same preconditions and effects, but application is different.

Action Applicability in Delete-Relaxed Space

An action $a = (\text{pre}_a, \text{eff}_a)$ is **applicable** in a delete-relaxed state s^+ if:

$$\text{pre}_a \subseteq s^+$$

This is the same as in the original FDR semantics: all preconditions must be present in the state.

State Transition in Delete-Relaxed Space

Super simple rule: Just ADD the new facts, never DELETE anything!
When an applicable action a is applied to a delete-relaxed state s^+, the successor state is:

$$s^+[a] = s^+ \cup \text{eff}_a$$

Read as: "New state = Old state + New effects" (like adding items to a shopping cart that you never remove!)
Key difference from original FDR:

- Original FDR: $s[a] = (s \setminus \{(v, d) \mid v \in \text{vars}(\text{eff}_a)\}) \cup \text{eff}_a$ (remove old values first, then add new)
- Delete-relaxed: $s^+[a] = s^+ \cup \text{eff}_a$ (just add new facts, never remove anything!)

Consequence: Facts accumulate monotonically. Once true, always true forever!
Visual metaphor: Think of it like a to-do list where you can only add checkmarks, never erase them - even if you "undo" the task, the checkmark stays!

Delete-Relaxed State Space Θ_Π^+

The **delete-relaxed state space** is the set of all reachable delete-relaxed states:

$$\Theta_\Pi^+ = \{s^+ \mid s^+ \text{ is reachable from } I^+ \text{ by applying delete-relaxed actions}\}$$

Properties:

- Θ_Π^+ is typically much smaller than Θ_Π because monotonicity reduces branching
- Every state in Θ_Π^+ is reachable from I^+ via a sequence of actions
- Goal states: $\{s^+ \in \Theta_\Pi^+ \mid G \subseteq s^+\}$

2.3 Delete-Relaxed Plans and Heuristic h^+

Delete-Relaxed Plan

A **delete-relaxed plan** is a sequence of actions $\pi^+ = \langle a_1, a_2, \ldots, a_k \rangle$ that:

1. Is executable from I^+ in the delete-relaxed state space
2. Reaches a goal state: $G \subseteq I^+[\pi^+]$

The cost of a delete-relaxed plan is:

$$\text{cost}(\pi^+) = \sum_{i=1}^{k} \text{cost}(a_i)$$

The h^+ Heuristic

The **optimal delete-relaxation heuristic** $h^+(s)$ is defined as:

$$h^+(s) = \min\{\text{cost}(\pi^+) \mid \pi^+ \text{ is a delete-relaxed plan from } s^+ \text{ to a goal}\}$$

That is, $h^+(s)$ is the cost of an *optimal* solution to the delete-relaxed problem starting from state s.

Special case: If no delete-relaxed plan exists, $h^+(s) = \infty$.

Admissibility of h^+

Theorem: The heuristic h^+ is **admissible**.
Proof intuition:

- Any plan π that solves the original problem also solves the delete-relaxed problem (because the relaxed problem has fewer constraints)
- Therefore, the optimal delete-relaxed plan cannot be more expensive than the optimal original plan
- Formally: $h^+(s) \leq h^*(s)$ for all states s

Consequence: Using h^+ in A* search guarantees finding optimal plans.

2.4 Computational Complexity of h^+

Computing h^+ is Hard

Problem: Although the delete-relaxed problem is easier than the original, computing h^+ exactly is still **NP-hard**.
Why? Even without deletes, finding the *optimal* relaxed plan requires exploring many action combinations.
Solution: Use *approximations* of h^+ that can be computed efficiently in polynomial time.

3 Computing Delete-Relaxation Heuristics

3.1 Relaxed Planning Graph (RPG)

To efficiently compute approximations of h^+, we use a **relaxed planning graph** (RPG). This is a layered structure that represents which facts can become true at which "time steps" in the delete-relaxed problem.

Relaxed Planning Graph Structure

A **relaxed planning graph** (RPG) for state s consists of alternating layers:
Fact layers: $F_0, F_1, F_2, \ldots$

- F_i = set of facts that can be true after i actions
- $F_0 = s$ (initial facts from state s)

Action layers: $A_0, A_1, A_2, \ldots$

- A_i = set of actions whose preconditions are satisfied in F_i

Construction rules:

1. Start with $F_0 = s$ (facts in current state)

2. For each layer i:
   - $A_i = \{a \in A \mid \text{pre}_a \subseteq F_i\}$ (all applicable actions)
   - $F_{i+1} = F_i \cup \bigcup_{a \in A_i} \text{eff}_a$ (accumulate new facts)
3. Stop when $G \subseteq F_i$ (goal is reached) or $F_{i+1} = F_i$ (fixed point)

Key property: Due to delete relaxation, facts are monotonic: $F_0 \subseteq F_1 \subseteq F_2 \subseteq \ldots$

RPG Example: Simple Logistics

Problem:

- Variables: $\textit{truck-loc} \in \{A, B\}$, $\textit{package-loc} \in \{A, B, \textit{in-truck}\}$
- Actions:
  - $\textit{drive_A_B}$: pre $= \{\textit{truck} = A\}$, eff $= \{\textit{truck} = B\}$, cost $= 1$
  - $\textit{drive_B_A}$: pre $= \{\textit{truck} = B\}$, eff $= \{\textit{truck} = A\}$, cost $= 1$
  - $\textit{load_A}$: pre $= \{\textit{truck} = A, \textit{pkg} = A\}$, eff $= \{\textit{pkg} = \textit{in-truck}\}$, cost $= 1$
  - $\textit{unload_B}$: pre $= \{\textit{truck} = B, \textit{pkg} = \textit{in-truck}\}$, eff $= \{\textit{pkg} = B\}$, cost $= 1$
- Initial state: $I = \{\textit{truck} = A, \textit{pkg} = A\}$
- Goal: $G = \{\textit{pkg} = B\}$

RPG Construction:
Layer 0 (Facts):

$$F_0 = \{\textit{truck} = A, \textit{pkg} = A\}$$

Layer 0 (Actions):

$$A_0 = \{\textit{drive_A_B}, \textit{load_A}\} \quad \text{(both have satisfied preconditions)}$$

Layer 1 (Facts):

$$F_1 = F_0 \cup \{\textit{truck} = B\} \cup \{\textit{pkg} = \textit{in-truck}\} = \{\textit{truck} = A, \textit{truck} = B, \textit{pkg} = A, \textit{pkg} = \textit{in-truck}\}$$

Note: $\textit{truck} = A$ and $\textit{truck} = B$ coexist due to delete relaxation!
Layer 1 (Actions):

$$A_1 = \{\textit{drive_A_B}, \textit{drive_B_A}, \textit{load_A}, \textit{unload_B}\} \quad \text{(all actions now applicable)}$$

Layer 2 (Facts):

$$F_2 = F_1 \cup \{\textit{pkg} = B\} = \{\textit{truck} = A, \textit{truck} = B, \textit{pkg} = A, \textit{pkg} = \textit{in-truck}, \textit{pkg} = B\}$$

Goal reached: $G = \{\textit{pkg} = B\} \subseteq F_2$ at layer 2.

3.2 Heuristic Estimates from RPG

The RPG provides several ways to estimate $h^+(s)$:

Goal Distance Heuristic

The **goal distance** is the first layer i where $G \subseteq F_i$:

$$h_{\text{level}}(s) = \min\{i \mid G \subseteq F_i\}$$

This counts the minimum number of action layers needed to reach the goal in the RPG.
Property: h_{level} is admissible but often weak (assumes unit costs).

Fact Cost Heuristics: $h^{\max}$ and h^{add}

For each fact f, define the **fact cost** $c(f)$ as an estimate of the minimum cost to achieve f from state s.
Initialization:

$$c(f) = \begin{cases} 0 & \text{if } f \in s \\ \infty & \text{otherwise} \end{cases}$$

Update rule: For each action a and each fact $f \in \text{eff}_a$:

$$c(f) \leftarrow \min\big(c(f), \text{cost}(a) + \text{aggregation}(\{c(f') \mid f' \in \text{pre}_a\})\big)$$

where the aggregation function determines the heuristic type:
1. Max heuristic ($h^{\max}$):

$$h^{\max}(s) = \max_{f \in G} c(f)$$

Uses: aggregation = max
Interpretation: Cost to achieve the hardest goal fact.
2. Additive heuristic (h^{add}):

$$h^{\text{add}}(s) = \sum_{f \in G} c(f)$$

Uses: aggregation = $\sum$
Interpretation: Sum of costs to achieve all goal facts independently.
Properties:

- $h^{\max}$ is admissible: $h^{\max}(s) \leq h^{+}(s) \leq h^{*}(s)$
- h^{add} is **not admissible** (can overestimate) but often more informed than $h^{\max}$

3.3 Simple Delete Relaxation Heuristic Algorithm

SimpleDeleteRelaxationHeur Algorithm (Layer-Based)

Algorithm 1 SimpleDeleteRelaxationHeur - Layer-Based Approach

```
function SimpleDeleteRelaxationHeur(state s)
    F ← s                                   ▷ Reached facts, initialized with input state
    C ← 0                                   ▷ Estimate of delete-relaxed plan cost
    𝒜 ← A                                   ▷ Actions not applied yet (all actions available)
    while G ⊈ F do                          ▷ Until all goal facts are reached
        𝒜' ← {a ∈ 𝒜 | pre_a ⊆ F}            ▷ Remaining applicable actions
        F' ← F ∪ ⋃_{a∈𝒜'} eff_a             ▷ Build next layer of reached facts
        if F = F' then
            return ∞                        ▷ No new facts reached, goal unreachable
        end if
        C ← C + min_{a∈𝒜'} cost(a)          ▷ Update heuristic with cheapest action cost
        𝒜 ← 𝒜 \ 𝒜'                          ▷ Update set of remaining actions
        F ← F'                              ▷ Switch to next layer
    end while
    return C                                ▷ Return total estimated cost
end function
```

3.3.1 HOW TO EXECUTE THIS ALGORITHM MANUALLY (FOR EXAMS!)

Step-by-Step Manual Execution Guide

SETUP - Before you start, write down:

1. List all your **variables and their domains**
2. List all **actions** with their preconditions, effects, and costs
3. Write down the **initial state** s
4. Write down the **goal** G

CREATE THIS TABLE FORMAT (you'll fill it row by row):

Layer	**Facts** F_i	**Goal reached?**	**Applicable Actions** $\mathcal{A}_i$	**Cost** C
0	(initial state)	Yes/No	(list actions)	0
1	(accumulated facts)	Yes/No	(list actions)	(cumulative)
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

EXECUTION STEPS - Repeat for each layer until goal is reached:
Step 1: Write down current facts F_i

- For layer 0: $F_0 = s$ (just copy the initial state)
- For layer $i > 0$: F_i = all facts from previous layer F_{i-1} PLUS new facts from actions

Step 2: Check if goal is reached

- Look at each fact in the goal G
- Check: Is every goal fact in F_i?
- If YES →
  textbfSTOP! Return cost C

- If NO → continue to Step 3

Step 3: Find applicable actions

- Go through each action in your action list
- For each action, check: Are ALL its preconditions in F_i?
- If YES → this action is applicable! Add it to $\mathcal{A}_i$
- If NO → skip this action

Step 4: Check for dead-end

- If NO actions are applicable AND goal not reached → textbfSTOP! Return ∞ (unreachable)

Step 5: Add cheapest action cost

- Look at all applicable actions in $\mathcal{A}_i$
- Find the one with the textbfminimum cost
- Update: $C = C + \min\{\text{cost}(a) \mid a \in \mathcal{A}_i\}$

Step 6: Generate new facts for next layer

- Start with: $F_{i+1} = F_i$ (copy all current facts)
- For EACH applicable action in $\mathcal{A}_i$:
  - Add ALL its effects to F_{i+1}
  - Remember: Delete relaxation means we NEVER remove facts!

Step 7: Move to next layer

- Increment layer counter: $i = i + 1$
- Go back to Step 1

Algorithm Explanation - Layer-Based Approach

Key Idea: Build layers of reachable facts, like breadth-first search.
How it works:

1. **Initialize:**
   - F = facts currently reached (start with state s)
   - C = cumulative cost estimate (starts at 0)
   - $\mathcal{A}$ = actions still available to use
2. **Build layers:** Repeat until goal G is reached:
   - Find all applicable actions $\mathcal{A}'$ (whose preconditions $\subseteq F$)
   - Build next fact layer F' by applying ALL applicable actions simultaneously
   - If no new facts added ($F = F'$), goal is unreachable → return ∞
   - Add cost of **cheapest** action in this layer to C

- Remove used actions from $\mathcal{A}$, move to next layer ($F \leftarrow F'$)

3. **Return:** The cumulative cost C is the heuristic estimate

Why "layer-based"?

- Each iteration builds a complete *layer* of facts (like BFS)
- All applicable actions fire simultaneously in each layer
- Cost per layer = minimum action cost (optimistic estimate)

Complexity: $O(|A| \cdot |V|)$ - polynomial in problem size.
Result: Computes an admissible (optimistic) heuristic estimate of the delete-relaxed plan cost.

WORKED EXAMPLE: SimpleDeleteRelaxationHeur with Tables

PROBLEM SETUP:
Initial state: $s = \{truck{=}A, pkg{=}A\}$
Goal: $G = \{pkg{=}B\}$
Available actions:

- $drive_A_B$: pre = $\{truck{=}A\}$, eff = $\{truck{=}B\}$, cost = 1
- $load_A$: pre = $\{truck{=}A, pkg{=}A\}$, eff = $\{pkg{=}in_truck\}$, cost = 1
- $unload_B$: pre = $\{truck{=}B, pkg{=}in_truck\}$, eff = $\{pkg{=}B\}$, cost = 1

EXECUTION TABLE (fill this in layer by layer):

Layer	**Facts F_i**	**Goal?**	**Applicable Actions**	**Cost C**
0	$\{truck{=}A, pkg{=}A\}$	No	$drive_A_B$, $load_A$	0
1	$\{truck = A, pkg = A, truck = B, pkg = in_truck\}$	No	$drive_A_B$, $load_A$, $unload_B$	$0 + 1 = 1$
2	$\{truck = A, pkg = A, truck = B, pkg = in_truck, pkg{=}B\}$	**YES!**	(stop here)	$1 + 1 = \mathbf{2}$

DETAILED STEP-BY-STEP WORK:
LAYER 0:

1. **Current facts:** $F_0 = \{truck{=}A, pkg{=}A\}$ (initial state)
2. **Goal check:** Is $pkg{=}B$ in F_0? NO $\rightarrow$ continue
3. **Find applicable actions:**
   - $drive_A_B$: pre = $\{truck{=}A\}$ ✓ (satisfied) $\rightarrow$ APPLICABLE
   - $load_A$: pre = $\{truck{=}A, pkg{=}A\}$ ✓ (both facts present) $\rightarrow$ APPLICABLE
   - $unload_B$: pre = $\{truck{=}B, pkg{=}in_truck\}$ $\times$ (not satisfied) $\rightarrow$ NOT applicable
4. **Applicable actions:** $\mathcal{A}_0 = \{drive_A_B, load_A\}$
5. **Update cost:** $C = 0 + \min\{1, 1\} = 1$
6. **Generate next layer facts:**
   - Start with F_0: $\{truck{=}A, pkg{=}A\}$
   - Add effects of $drive_A_B$: $\{truck{=}B\}$
   - Add effects of $load_A$: $\{pkg{=}in_truck\}$

- Result: $F_1 = \{truck{=}A, pkg{=}A, truck{=}B, pkg{=}in_truck\}$

LAYER 1:

1. **Current facts:** $F_1 = \{truck{=}A, pkg{=}A, truck{=}B, pkg{=}in_truck\}$
2. **Goal check:** Is $pkg{=}B$ in F_1? NO $\rightarrow$ continue
3. **Find applicable actions:**
   - $drive_A_B$: pre $= \{truck{=}A\}$ ✓ $\rightarrow$ APPLICABLE
   - $load_A$: pre $= \{truck{=}A, pkg{=}A\}$ ✓ $\rightarrow$ APPLICABLE
   - $unload_B$: pre $= \{truck{=}B, pkg{=}in_truck\}$ ✓ (NOW both present!) $\rightarrow$ APPLICABLE
4. **Applicable actions:** $\mathcal{A}_1 = \{drive_A_B, load_A, unload_B\}$
5. **Update cost:** $C = 1 + \min\{1, 1, 1\} = 2$
6. **Generate next layer facts:**
   - Start with F_1
   - Add effects of $unload_B$: $\{pkg{=}B\}$ (other actions add no new facts)
   - Result: $F_2 = \{truck{=}A, pkg{=}A, truck{=}B, pkg{=}in_truck, pkg{=}B\}$

LAYER 2:

1. **Current facts:** $F_2 = \{truck{=}A, pkg{=}A, truck{=}B, pkg{=}in_truck, pkg{=}B\}$
2. **Goal check:** Is $pkg{=}B$ in F_2? **YES!** $\rightarrow$ **STOP!**

FINAL ANSWER: $h(s) = C = \mathbf{2}$

KEY OBSERVATIONS:

- Notice how $truck{=}A$ and $truck{=}B$ coexist from layer 1 onwards (delete relaxation!)
- We counted the minimum cost at each layer (1 + 1 = 2)
- This gives an optimistic estimate (admissible heuristic)

4 The FF Heuristic

4.1 Motivation for FF

The problem with $h^{\max}$: While it's safe (admissible), it's often too pessimistic - like a GPS that says "the destination is at least 50 miles away" when it's actually 51 miles. Not very helpful!

While $h^{\max}$ is admissible, it can be too conservative (underestimates significantly). The **FF heuristic** (Fast Forward) aims to provide a more informed estimate by extracting an actual relaxed plan from the RPG.

What makes FF special? Instead of just computing abstract costs, FF actually builds a concrete plan in the relaxed world, then counts how many actions it used!

Key Idea of FF Heuristic

The Recipe (3 steps):
The FF heuristic:

1. Constructs a relaxed planning graph (RPG) forward from the current state
   - **Think:** Build layers showing what becomes possible at each step

2. Performs **backward search** through the RPG to extract a concrete relaxed plan
   - **Think:** Work backwards from goal to find which actions we actually need
3. Returns the *length* (number of actions) of this relaxed plan as the heuristic value
   - **Think:** Count how many steps the plan took

Advantages:

- More informed than $h^{\max}$ (often closer to h^+)
- Still computable in polynomial time
- Produces an actual plan (useful for debugging and analysis)

Disadvantage:

- **Not admissible** in general (can overestimate due to backward chaining heuristics)

4.2 FF Algorithm Structure

Overview: FF is like planning a trip - first you figure out what's reachable (Phase 1), then you work backwards to find the actual route (Phase 2)!

The FF heuristic consists of two phases:

Phase 1: Forward RPG Construction - "What Can I Reach?"

Goal: Build the relaxed planning graph until all goal facts appear.
Simple analogy: Like filling in a map showing "after 1 step, I can reach these cities; after 2 steps, these cities too..."
Process:

1. Initialize $F_0 = s$ - Start with what you currently have
2. For $i = 0, 1, 2, \ldots$ (each time step):
   - Compute $A_i = \{a \in A \mid \text{pre}_a \subseteq F_i\}$ - Find all actions you can do now
   - Compute $F_{i+1} = F_i \cup \bigcup_{a \in A_i} \text{eff}_a$ - Add all facts these actions give you
   - Track the *earliest layer* where each fact first appears - Remember when you first got each fact
   - Stop when $G \subseteq F_i$ - Stop when you have everything you need for the goal!

Output: Layered RPG structure with fact achievement levels (a roadmap of possibilities).

Phase 2: Backward Relaxed Plan Extraction - "What Did I Actually Need?"

Goal: Extract a relaxed plan by working backward from the goal.
Simple analogy: Like tracing your steps backward - "To be at the goal, I needed X; to get X, I needed Y; to get Y, I needed Z..."
Process:

1. Start with goal layer i where $G \subseteq F_i$ - Start at the end (the goal)
2. Initialize subgoals: $SG_i = G$ - These are the things we need
3. For $j = i, i-1, \ldots, 1$ (walk backwards through layers):

- For each fact $f \in SG_j$ that is not in F_{j-1} (facts we don't have yet):
  - Choose an action $a \in A_{j-1}$ that achieves f (i.e., $f \in \text{eff}_a$) - Pick an action that gives us f
  - Add a to the relaxed plan - Remember this action
  - Add preconditions of a to SG_{j-1}: $SG_{j-1} \leftarrow SG_{j-1} \cup \text{pre}_a$ - Now we need the action's requirements too!

Output: A set of actions forming a relaxed plan (the actions we need to execute).
Heuristic value: $h^{\text{FF}}(s)$ = number of actions in the extracted plan (how many steps we found).

4.3 FF Algorithm - Detailed Pseudocode

FF Heuristic Algorithm

Algorithm 2 FF Heuristic

```
function FFHeuristic(Π, s)
    Π = (V, A, cost, I, G)
    Phase 1: Forward RPG Construction
    F[0] ← s
    level[f] ← 0 for all f ∈ s                          ▷ Track when facts first appear
    level[f] ← ∞ for all f ∉ s
    i ← 0
    while G ⊈ F[i] do
        A[i] ← {a ∈ A | pre_a ⊆ F[i]}
        if A[i] = ∅ then
            return ∞                                    ▷ Goal unreachable
        end if
        F[i + 1] ← F[i]
        for each a ∈ A[i] do
            for each f ∈ eff_a do
                if f ∉ F[i] then
                    F[i + 1] ← F[i + 1] ∪ {f}
                    level[f] ← i + 1
                end if
            end for
        end for
        i ← i + 1
    end while
    goal_layer ← i                                      ▷ Layer where goal is achieved

    Phase 2: Backward Plan Extraction
    plan ← ∅                                            ▷ Actions in relaxed plan
    SG[goal_layer] ← G                                  ▷ Subgoals at goal layer
    for j = goal_layer down to 1 do
        SG[j − 1] ← ∅
        for each fact f ∈ SG[j] do
            if level[f] < j then                        ▷ Fact already achieved in earlier layer
                Continue                                ▷ No action needed
            end if
            Choose action a ∈ A[j − 1] where f ∈ eff_a  ▷ Select achiever
            plan ← plan ∪ {a}
            SG[j − 1] ← SG[j − 1] ∪ pre_a               ▷ Add preconditions as subgoals
        end for
    end for
    return |plan|                                       ▷ Number of actions in relaxed plan
end function
```

4.3.1 HOW TO EXECUTE FF HEURISTIC MANUALLY (FOR EXAMS!)

Manual FF Execution Guide - COMPLETE PROCESS

========== PHASE 1: FORWARD RPG CONSTRUCTION ==========

SETUP - What you need to write down:

1. Problem definition (variables, actions with pre/eff, initial state, goal)
2. A **FACTS TABLE** to track when each fact first appears
3. A **LAYERS TABLE** to build the RPG

FACTS TABLE - Create this first (track when facts appear):

Fact	First appears at layer	Notes
(fact 1)	0 (or layer #)	(how it was achieved)
⋮	⋮	⋮

RPG LAYERS TABLE - Fill layer by layer:

Layer i	Facts $F[i]$	Applicable Actions $A[i]$
0	(initial state)	(actions with pre $\subseteq F[0]$)
1	$F[0]$ + new facts from $A[0]$	(actions with pre $\subseteq F[1]$)
⋮	⋮	⋮

FORWARD PHASE STEPS (repeat until goal reached):
Step 1: Initialize Layer 0

- $F[0] = s$ (initial state)
- For each fact $f \in F[0]$: write $level[f] = 0$ in your facts table
- For all other facts: $level[f] = \infty$

Step 2: Check Goal

- Is every goal fact in $F[i]$?
- If YES $\rightarrow$ record $goal_layer = i$ and go to PHASE 2
- If NO $\rightarrow$ continue

Step 3: Find Applicable Actions

- For each action: check if ALL preconditions are in $F[i]$
- List all applicable actions in $A[i]$ column

Step 4: Build Next Layer

- Start: $F[i+1] = F[i]$ (copy all current facts)
- For EACH action in $A[i]$:
  - For EACH effect f of the action:
  - If $f \notin F[i]$ (NEW fact):

- Add f to $F[i+1]$
- Record $level[f] = i+1$ in facts table

Step 5: Move to next layer and repeat from Step 2

========== PHASE 2: BACKWARD PLAN EXTRACTION ==========

SUBGOALS TABLE - Create this for backward phase:

Layer	**Subgoals** $SG[j]$	**New facts needing achievers**	**Actions chosen**
$goal_layer$	G (goal facts)	(facts with $level = goal_layer$)	(actions chosen)
$goal_layer - 1$	(preconditions of chosen actions)	(facts with $level = goal_layer - 1$)	(actions chosen)
$\vdots$	$\vdots$	$\vdots$	$\vdots$
0	(should all be in $F[0]$!)	(none)	(done!)

BACKWARD PHASE STEPS (work backwards from goal):
Step 1: Initialize at goal layer

- $SG[goal_layer] = G$ (all goal facts)
- $plan = \emptyset$ (empty plan initially)

Step 2: For each layer j from $goal_layer$ down to 1:

1. Initialize: $SG[j-1] = \emptyset$
2. For EACH fact f in $SG[j]$:
   - **Check:** Is $level[f] < j$? (fact achieved earlier?)
   - If YES $\rightarrow$ skip this fact (already achieved!)
   - If NO $\rightarrow$ this fact needs an achiever action:
     (a) Look at layer $j-1$ actions: find action $a \in A[j-1]$ where $f \in \text{eff}_a$
     (b) Add a to $plan$
     (c) Add ALL preconditions of a to $SG[j-1]$: $SG[j-1] = SG[j-1] \cup \text{pre}_a$

Step 3: Count actions in plan

- Final answer: $h^{FF}(s) = |plan|$ (number of actions chosen)

COMMON MISTAKES TO AVOID:

- **X** Forgetting that facts accumulate (delete relaxation!)
- **X** Not checking if a fact was already achieved in earlier layer
- **X** Forgetting to add preconditions as subgoals in backward phase
- **X** Counting duplicate actions (each action should appear once in plan)

FF Algorithm Notes

Key decisions in FF:

1. **Action selection:** When multiple actions can achieve a fact, FF typically chooses the one that appears earliest in the RPG or uses other heuristics
2. **Greedy extraction:** The backward phase is greedy and doesn't optimize the plan
3. **Unit costs:** FF counts actions (assumes unit cost). For non-unit costs, variants use action costs.

Complexity: $O(|A| \cdot |V|^2)$ - polynomial and practical.

4.4 FF Heuristic Example: Complete Worked Example with Tables

COMPLETE FF EXAMPLE: Logistics Problem

PROBLEM SETUP:

- **Variables:** $truck\text{-}loc \in \{A, B, C\}$, $pkg\text{-}loc \in \{A, B, C, in\text{-}truck\}$
- **Initial:** $I = \{truck = A, pkg = A\}$
- **Goal:** $G = \{pkg = C\}$

Actions (simplified for space):

- $drive_A_C$: pre $= \{truck = A\}$, eff $= \{truck = C\}$
- $load_A$: pre $= \{truck = A, pkg = A\}$, eff $= \{pkg = in\text{-}truck\}$
- $unload_C$: pre $= \{truck = C, pkg = in\text{-}truck\}$, eff $= \{pkg = C\}$

PHASE 1: FORWARD RPG CONSTRUCTION
Facts Tracking Table:

Fact	Level	How achieved
$truck = A$	0	Initial state
$pkg = A$	0	Initial state
$truck = C$	1	Effect of $drive_A_C$
$pkg = in\text{-}truck$	1	Effect of $load_A$
$pkg = C$	2	Effect of $unload_C$

RPG Layers Table:

Layer	Facts $F[i]$	Applicable Actions $A[i]$
0	$\{truck = A, pkg = A\}$	$drive_A_C$, $load_A$
1	$\{truck = A, pkg = A, truck = C, pkg = in\text{-}truck\}$	$drive_A_C$, $load_A$, $unload_C$
2	$\{truck = A, pkg = A, truck = C, pkg = in\text{-}truck,$ **pkg=C**$\}$	Goal reached!

Goal reached at layer 2! Now start backward extraction.

PHASE 2: BACKWARD PLAN EXTRACTION
Subgoals Tracking Table (work backwards):

Layer	Subgoals $SG[j]$	Facts needing achievers	Actions added to plan
2	$\{pkg = C\}$ (goal)	$pkg = C$ (level=2)	$unload_C$
1	$\{truck = C, pkg = in\text{-}truck\}$	$truck = C$ (level=1), $pkg = in\text{-}truck$ (level=1)	$drive_A_C$, $load_A$
0	$\{truck = A, pkg = A\}$	None (all in $F[0]$)	Done!

DETAILED BACKWARD WORK:
At Layer 2:

- $SG[2] = \{pkg = C\}$ (the goal)
- Check $pkg = C$: $level[pkg = C] = 2$, so it's NEW at layer 2
- Need achiever: Choose $unload_C$ from $A[1]$ (has $pkg = C$ in effects)
- Add $unload_C$ to plan
- Add preconditions of $unload_C$ to $SG[1]$: $SG[1] = \{truck = C, pkg = in\text{-}truck\}$

At Layer 1:

- $SG[1] = \{truck = C, pkg = in\text{-}truck\}$
- Check $truck = C$: $level[truck = C] = 1$, so it's NEW at layer 1
  - Choose $drive_A_C$ from $A[0]$ (has $truck = C$ in effects)
  - Add $drive_A_C$ to plan
  - Add preconditions: $\{truck = A\}$ to $SG[0]$
- Check $pkg = in\text{-}truck$: $level[pkg = in\text{-}truck] = 1$, so it's NEW at layer 1
  - Choose $load_A$ from $A[0]$ (has $pkg = in\text{-}truck$ in effects)
  - Add $load_A$ to plan
  - Add preconditions: $\{truck = A, pkg = A\}$ to $SG[0]$
- Result: $SG[0] = \{truck = A, pkg = A\}$

At Layer 0:

- $SG[0] = \{truck = A, pkg = A\}$
- All facts in $SG[0]$ have $level = 0$ (already in initial state!)
- No actions needed. Done!

EXTRACTED RELAXED PLAN:

$$\text{plan} = \{drive_A_C, load_A, unload_C\}$$

FF HEURISTIC VALUE: $h^{\text{FF}}(I) = |\text{plan}| = \mathbf{3}$

INTERPRETATION:

- FF estimates we need 3 actions to reach the goal
- The actual optimal plan is: $\langle load_A, drive_A_C, unload_C\rangle$ (also cost 3!)
- In this case, FF gives an accurate estimate

4.5 FF Heuristic Example: Blocks World

FF Example: Blocks World

Problem: Stack blocks A, B, C on a table.
Variables:

- $on(A) \in \{B, C, table\}$
- $on(B) \in \{A, C, table\}$
- $on(C) \in \{A, B, table\}$
- $clear(X) \in \{true, false\}$ for $X \in \{A, B, C\}$
- $holding \in \{A, B, C, none\}$

Initial state:

$$I = \{on(A) = table, on(B) = table, on(C) = A, clear(B) = true, clear(C) = true, holding = none\}$$

Goal:

$$G = \{on(A) = B, on(B) = C\}$$

(Want: C on table, B on C, A on B)
Actions: $pickup(X)$, $putdown(X)$, $stack(X, Y)$, $unstack(X, Y)$
Forward RPG Construction:
Layer 0: Initial facts
Layer 0 actions:

- $unstack(C, A)$: pre $= \{on(C) = A, clear(C) = true, holding = none\}$, eff $= \{holding = C, clear(A) = true\}$
- $pickup(B)$: pre $= \{on(B) = table, clear(B) = true, holding = none\}$, eff $= \{holding = B\}$

Layer 1: New facts: $\{holding = C, clear(A) = true, holding = B\}$
Note: In delete-relaxed world, both $holding = C$ and $holding = B$ can be true!
Continue forward... until $\{on(A) = B, on(B) = C\}$ appears.
Backward extraction: Work backward to find actions that achieve $on(A) = B$ and $on(B) = C$, ultimately extracting a relaxed plan.
Example relaxed plan:

$$\pi^+ = \{unstack(C, A), putdown(C), pickup(B), \ldots, stack(A, B)\}$$

FF value: $h^{\mathrm{FF}}(I) =$ length of extracted plan (e.g., 5 actions).

4.6 Properties of FF Heuristic

FF Heuristic Properties

Advantages:

1. **Efficient:** Computable in polynomial time $O(|A| \cdot |V|^2)$
2. **Informed:** Often much closer to h^+ than $h^{\max}$
3. **Practical:** Works very well in practice for many planning domains

4. **Produces plans:** Extracts actual relaxed plans (useful for analysis)

Disadvantages:

1. **Not admissible:** Can overestimate h^+, so A* with FF may miss optimal solutions
2. **Not consistent:** FF values can be non-monotonic
3. **Greedy extraction:** Backward phase doesn't optimize, so extracted plan may not be optimal even in relaxed space

Use cases:

- **Satisficing planning:** When finding any solution quickly is more important than optimality
- **Greedy best-first search:** Use FF with GBFS (not A*) for fast solutions

4.7 Why FF is Not Admissible

The catch: FF makes greedy choices when extracting the plan, and greedy doesn't always mean optimal!

Simple explanation: Imagine you need to buy bread and milk. FF might greedily say "go to store A for bread (2 miles), then store B for milk (2 miles) = 4 miles total." But there's a better option: "go to store C that has both (3 miles)!" FF missed it because it wasn't looking for the globally best solution, just making quick local choices.

FF Non-Admissibility Example

Consider a simplified example where:
Original problem:

- Optimal plan has 2 actions
- Goal requires achieving facts f_1 and f_2

Relaxed plan extraction (what FF does):

- FF backward extraction greedily selects actions (picks the first action it sees for each fact)
- It might choose action a to achieve f_1 and action b to achieve f_2
- Both actions together have preconditions that require 3 actions total in the relaxed plan
- **FF's answer:** "You need 3 actions"

But: There exists a different relaxed plan using a single action c that achieves both f_1 and f_2 with only 2 actions total.

- Action c was smarter - it achieved both goals at once!
- **True answer:** "You only need 2 actions"

Result: FF extracts a 3-action plan, while the optimal relaxed plan has 2 actions.
Conclusion: $h^{\text{FF}} = 3 > h^+ = 2$, violating admissibility (FF overestimated!)
Why this happens: FF's backward extraction is greedy - it quickly picks actions without checking if there's a better combination. Speed vs. accuracy trade-off!

5 Additional Delete-Relaxation Heuristics

5.1 Comparison of Heuristics

Heuristic	Admissible?	Computation	Informativeness
h^+	Yes	NP-hard	Optimal relaxed
$h^{\max}$	Yes	Polynomial	Conservative
h^{add}	No	Polynomial	More informed
h^{FF}	No	Polynomial	Very informed

Heuristic Relationships

Dominance hierarchy:

$$h^{\max}(s) \leq h^+(s) \leq h^*(s)$$

In practice:

$$h^{\max}(s) \leq h^+(s) \lesssim h^{\text{FF}}(s) \quad \text{(FF can exceed } h^+\text{)}$$

General trend:

$$\text{Informativeness: } h^{\max} < h^{\text{add}} < h^{\text{FF}} \quad \text{(but with loss of admissibility)}$$

5.2 LM-Cut Heuristic (Advanced)

LM-Cut: Admissible and Informed

The **LM-Cut heuristic** is a more sophisticated admissible heuristic that improves upon $h^{\max}$:
Key idea:

- Identify **landmarks**: facts that must appear in any plan
- Find **disjunctive action landmarks**: sets of actions where at least one must be used
- Compute a "cut" in the relaxed planning graph
- Iteratively extract and sum the costs of cuts

Properties:

- **Admissible:** $h^{\text{LM-Cut}}(s) \leq h^+(s) \leq h^*(s)$
- **More informed than $h^{\max}$:** $h^{\max}(s) \leq h^{\text{LM-Cut}}(s)$
- **More expensive:** Higher computational cost than $h^{\max}$ or FF

Use case: Optimal planning with a strong admissible heuristic.

5.3 Choosing the Right Heuristic

Scenario	Recommended Heuristic
Need optimal plan, willing to wait	$h^{\text{LM-Cut}}$ or $h^{\max}$ with A*
Need fast satisficing plan	h^{FF} with GBFS
Want balance: reasonably fast, good quality	h^{add} or FF with weighted A*
Simple, guaranteed admissible	$h^{\max}$ with A*
Research/debugging relaxed plans	h^{+} (exact, but slow)

6 Practical Considerations and Extensions

6.1 Weighted A* with Non-Admissible Heuristics

Since heuristics like FF and h^{add} are not admissible but very informative, they are often used with **weighted A***:

Weighted A*

Standard A*: $f(n) = g(n) + h(n)$
Weighted A*: $f(n) = g(n) + w \cdot h(n)$ for weight $w \geq 1$
Effect:

- $w > 1$: Emphasizes heuristic more, speeds up search, but may sacrifice optimality
- $w = 1$: Standard A*
- Bounded suboptimality: Solution cost $\leq w \cdot$ optimal cost

Common choice: $w = 2$ or $w = 5$ for fast satisficing planning.

6.2 Using FF in Practice: The FF Planner

The **FF planner** (Fast Forward, developed by Jörg Hoffmann) is one of the most successful classical planners:

FF Planner Architecture

Search algorithm: Enforced Hill-Climbing (EHC) with FF heuristic

1. Compute h^{FF} for current state s
2. Perform breadth-first search to find a state s' with $h^{\text{FF}}(s') < h^{\text{FF}}(s)$
3. Move to s' and repeat
4. If no improvement found, switch to best-first search

Advantages:

- Very fast for many benchmark domains
- Finds solutions quickly without exploring huge search spaces

Domains where FF excels:

- Logistics
- Blocksworld
- Grid navigation
- Rovers

6.3 Limitations of Delete Relaxation

When Delete Relaxation Fails

Delete relaxation is less effective when:

1. **Resource constraints are critical:**
   - Example: Limited fuel in logistics
   - Relaxation ignores resource depletion
2. **Mutexes matter:**
   - Example: Only one object can be in a location
   - Relaxation allows impossible states (truck in multiple places)
3. **Negative interactions:**
   - Example: Actions that undo each other's effects
   - Relaxation doesn't capture this conflict
4. **Long-term planning:**
   - Example: Need to set up conditions far in advance
   - Relaxation gives overly optimistic short-term estimates

Alternative approaches:

- Pattern databases (abstraction-based heuristics)
- Critical-path heuristics
- Landmark-based heuristics (like LM-Cut)
- Potential heuristics (numeric planning)

6.4 Extensions: Beyond Simple Delete Relaxation

Advanced Techniques

1. **Red-Black Planning:**
   - Partition variables into "red" (relaxed, no deletes) and "black" (respected, with deletes)
   - Combine the benefits of both worlds
2. **Partial Delete Relaxation:**
   - Relax only some delete effects, not all
   - Trade-off between informativeness and computational cost
3. **Action-Cost-Based Relaxation:**
   - For non-unit costs, weight actions by their costs in FF extraction
   - Improves FF for cost-optimal planning
4. **Multi-Heuristic Planning:**
   - Use multiple heuristics ($h^{\max}$, FF, LM-Cut) in combination
   - Switch between heuristics based on search progress

7 Summary and Key Takeaways

7.1 Main Concepts

Essential Points to Remember

1. **Delete Relaxation:**
   - **Idea:** Ignore delete effects; facts stay true once achieved
   - **Result:** Easier planning problem (polynomial time solvable)
   - **State space:** Θ_Π^+ with monotonic fact accumulation
2. **Optimal Delete-Relaxation Heuristic h^+:**
   - $h^+(s)$ = cost of optimal delete-relaxed plan from s
   - **Admissible:** $h^+(s) \leq h^*(s)$
   - **NP-hard to compute exactly**
3. **Approximations of h^+:**
   - $h^{\max}$**:** Max cost among goal facts (admissible, polynomial)
   - $h^{\mathbf{add}}$**:** Sum of costs of goal facts (not admissible, but informed)
   - $h^{\mathbf{FF}}$**:** Length of extracted relaxed plan (not admissible, very informed)

- **LM-Cut:** Landmark-based admissible heuristic (stronger than $h^{\max}$)

4. **FF Heuristic Algorithm:**
   1. **Forward:** Build relaxed planning graph (RPG) until goal appears
   2. **Backward:** Extract relaxed plan by greedy backward chaining
   3. **Return:** Number of actions in extracted plan
5. **Properties:**
   - FF is **not admissible** but very effective in practice
   - Used in satisficing planning with GBFS or weighted A*
   - FF planner won many International Planning Competitions

7.2 Formulas and Notation Reference

Key Formulas

Delete-Relaxed State Transition:

$$s^+[a] = s^+ \cup \text{eff}_a$$

Optimal Delete-Relaxation Heuristic:

$$h^+(s) = \min_{\pi^+} \text{cost}(\pi^+) \quad \text{where } \pi^+ \text{ is a delete-relaxed plan from } s$$

Max Heuristic:

$$h^{\max}(s) = \max_{f \in G} c(f)$$

Additive Heuristic:

$$h^{\text{add}}(s) = \sum_{f \in G} c(f)$$

FF Heuristic:

$$h^{\text{FF}}(s) = \text{number of actions in extracted relaxed plan}$$

Admissibility Relationship:

$$h^{\max}(s) \leq h^+(s) \leq h^*(s)$$

7.3 Algorithm Complexity Summary

Algorithm	Time Complexity	Space Complexity
h^+ (exact)	NP-hard	Exponential
$h^{\max}$	$O(\|A\| \cdot \|V\|)$	$O(\|V\|)$
h^{add}	$O(\|A\| \cdot \|V\|)$	$O(\|V\|)$
h^{FF}	$O(\|A\| \cdot \|V\|^2)$	$O(\|A\| \cdot \|V\|)$
LM-Cut	$O(\|A\|^2 \cdot \|V\|^2)$	$O(\|A\| \cdot \|V\|)$

7.4 Practical Guidelines

When to Use Each Heuristic

Use $h^{\max}$ when:

- You need an admissible heuristic for optimal planning
- Simple, fast computation is important
- Acceptable to have a somewhat weak heuristic

Use $h^{\mathbf{FF}}$ when:

- Satisficing planning (don't need optimality)
- Want fast solutions
- Using GBFS or enforced hill-climbing

Use LM-Cut when:

- Optimal planning is required
- Need stronger heuristic than $h^{\max}$
- Can afford higher computation time

Use $h^{\mathbf{add}}$ when:

- Want more informed than $h^{\max}$
- Willing to sacrifice admissibility
- Using weighted A*

7.5 Connection to Previous and Future Topics

Previous topics (Lecture 4, Part 1):

- FDR representation: variables, actions, preconditions, effects
- State space Θ_Π and plans
- Complexity: PSPACE-complete

This section (Slides 18-42):

- Delete relaxation technique
- h^+, $h^{\max}$, h^{add}, h^{FF} heuristics
- Relaxed planning graphs
- FF algorithm and planner

Future topics:

- Pattern database heuristics (abstraction-based)

- Merge-and-shrink abstractions
- Partial-order planning
- HTN (Hierarchical Task Network) planning
- Temporal and numeric planning

End of Lecture 4: Delete Relaxation and FF Heuristic

This concludes the comprehensive coverage of slides 18-42.

These notes cover the theoretical foundations, algorithms, examples, and practical applications of delete-relaxation heuristics in classical planning.